

Netflix System Design

High-Level Design (HLD) Report — Rewritten Version

1. Introduction

Netflix operates as a worldwide video-on-demand service that enables subscribers to explore, play, and organize films, series, documentaries, and other video titles on a wide range of devices. The central engineering challenge is to start playback rapidly and sustain smooth, high-definition streaming with negligible rebuffering for tens of millions of simultaneous viewers spread across different regions.

This High-Level Design presents a scalable blueprint for a Netflix-style streaming system. The architecture is divided into two distinct planes: a control plane that manages identities, catalog metadata, personalization, subscriptions, and playback authorization, and a data plane that focuses purely on moving video bytes to viewers efficiently through a global Content Delivery Network (CDN).

2. Functional Requirements

- Account lifecycle support, including sign-up, sign-in, authentication, and management of users and profiles.
- Subscription handling and entitlement checks to control which plans can access which content.
- Content discovery through browsing of catalog rows and low-latency search across movies and series.
- Personalized suggestions driven by watch history, ratings, and stated preferences.
- Rich title details for every asset, such as synopsis, genre, cast and crew, runtime, maturity rating, and scores.
- Playback on browsers, smartphones, tablets, smart TVs, streaming sticks, and game consoles.
- Adaptive bitrate delivery that automatically raises or lowers quality as network throughput changes.
- Cross-device resume, allowing a viewer to continue exactly where the previous session stopped.
- Persistent viewing history together with an automatically maintained Continue Watching rail.
- A personal My List / Watchlist that users can curate by saving or removing titles.
- Multiple language dubs, subtitle tracks, closed captions, and selectable quality levels per title.

- Protected streaming using tokenized URLs, encryption, and DRM license enforcement.
- Collection of client telemetry, including playback events, error rates, and quality-of-experience signals.
- Studio-facing ingestion tools for uploading masters, attaching metadata, and publishing or retiring titles.

3. Non-Functional Requirements

- High availability: the service must stay reachable even if individual microservices, racks, or regions become unhealthy.
- Elastic scalability: the platform should absorb millions of concurrent sessions and a continuously expanding library without redesign.
- Responsive APIs: catalog, search, and session calls should normally return within a few hundred milliseconds.
- Fast time-to-first-frame: the interval between pressing Play and seeing video should be as short as possible.
- Massive throughput: the delivery path must move hundreds of terabits per second of video during peak hours.
- Durability: profiles, subscriptions, catalog records, and resume positions must never be lost after acknowledgement.
- Fault isolation: timeouts, retries with backoff, bulkheads, failover, and graceful degradation should contain failures locally.
- Global locality: viewers should be served from the closest healthy edge cache whenever feasible.
- End-to-end security: strong authentication, fine-grained authorization, encrypted transport, DRM, and safe session handling.
- Full observability: centralized logs, metrics, distributed traces, dashboards, and proactive alerting for every tier.
- Operability: each service should be independently built, deployed, versioned, owned, and scaled behind a well-defined contract.

4. Assumptions

- The service supports roughly 300 million registered accounts worldwide.
- About 100 million of those accounts are active on an ordinary day.
- Peak concurrency reaches approximately 30 million simultaneous streams.
- A typical daily viewer consumes close to 2 hours of content per day.
- The blended average delivered bitrate across mobile, HD, and 4K renditions is taken as 5 Mbps.
- The catalog holds on the order of 50,000 distinct movies and show episodes.

- Each ingested master is encoded into a ladder of resolutions and bitrates for adaptive playback.
- Almost all video bytes are offloaded to edge caches; origin servers rarely serve clients directly.
- Metadata, search, and recommendation traffic is orders of magnitude smaller than video egress.
- Playback heartbeats and interaction events flow through asynchronous pipelines so they never block streaming.

5. Back-of-the-Envelope Estimation

5.1 Peak Concurrent Bandwidth

Consider 30 million concurrent playbacks at a blended rate of 5 Mbps per stream.

Aggregate throughput = $30,000,000 \times 5 \text{ Mbps} = 150,000,000 \text{ Mbps} \approx 150 \text{ Tbps}$.

The delivery tier must therefore sustain on the order of 150 Tbps at global peak. This load cannot pass through core application servers and must be absorbed almost entirely by the CDN edge footprint.

5.2 Aggregate Daily Viewing

Consider 100 million daily viewers, each watching for 2 hours on average.

Total viewing = $100,000,000 \times 2 \text{ hours} = 200 \text{ million watched hours per day}$.

At a 5 Mbps mean bitrate this translates into an enormous volume of bytes moved every day, which justifies aggressive edge caching, adaptive ladders, efficient replication, and tiered origin storage.

5.3 Resume-Position Write Rate

Suppose every active stream checkpoints its position every 30 seconds, with 30 million streams in flight.

Write rate = $30,000,000 \div 30 \text{ s} \approx 1,000,000 \text{ writes per second}$.

Such a rate cannot hit a relational row on every heartbeat. In practice the system relies on client-side batching, sampling, buffering, write-behind caches, and asynchronous event consumers to protect the system of record.

5.4 Library Footprint

Suppose the catalog contains 50,000 titles with an average mezzanine size near 100 GB, each expanded into several renditions.

Baseline capacity = $50,000 \times 100 \text{ GB} = 5 \text{ PB}$ for source-grade copies alone. Once HLS/DASH segments, multiple codecs, audio dubs, subtitle bundles, thumbnails, and replicated copies are included, the total easily extends into the tens of petabytes.

6. System Components

The platform is decomposed into focused services and infrastructure blocks. Each block owns a narrow responsibility and can scale on its own trajectory:

Component	Responsibility
Client Applications	Applications for web, iOS/Android, smart TVs, and streaming devices. Responsible for UI, trick-play controls, adaptive-bitrate decisions, offline downloads, and API integration.
API Gateway	Unified ingress for all client traffic. Performs routing, throttling, authentication checks, schema validation, and response aggregation.
Authentication & User Service	Owns identities, credentials, sessions, profiles, parental controls, and registered-device records.
Subscription & Entitlement Service	Owns plans, billing state, and access rules that decide whether a given profile may play a given asset.
Content Catalog Service	System of record for discoverable metadata: titles, seasons, artwork, synopses, genres, cast, maturity labels, and regional availability windows.
Search Service	Maintains an inverted index over the catalog to answer full-text, faceted, and typo-tolerant queries within milliseconds.
Recommendation Service	Builds personalized home rows and rankings from viewing signals, collaborative filtering, embeddings, and ranking models.
Playback Service	Orchestrates a viewing session: entitlement verification, manifest selection, CDN URL signing, and DRM authorization.
Video Ingestion Pipeline	Accepts studio masters, runs QC checks, extracts metadata, and launches encoding workflows.
Transcoding Service	Produces the adaptive ladder — multiple resolutions, bitrates, and codecs — plus thumbnails, previews, and encrypted segments.
Object Storage	Durable store for masters, encoded chunks, manifests, captions, and image assets feeding both origin and CDN.

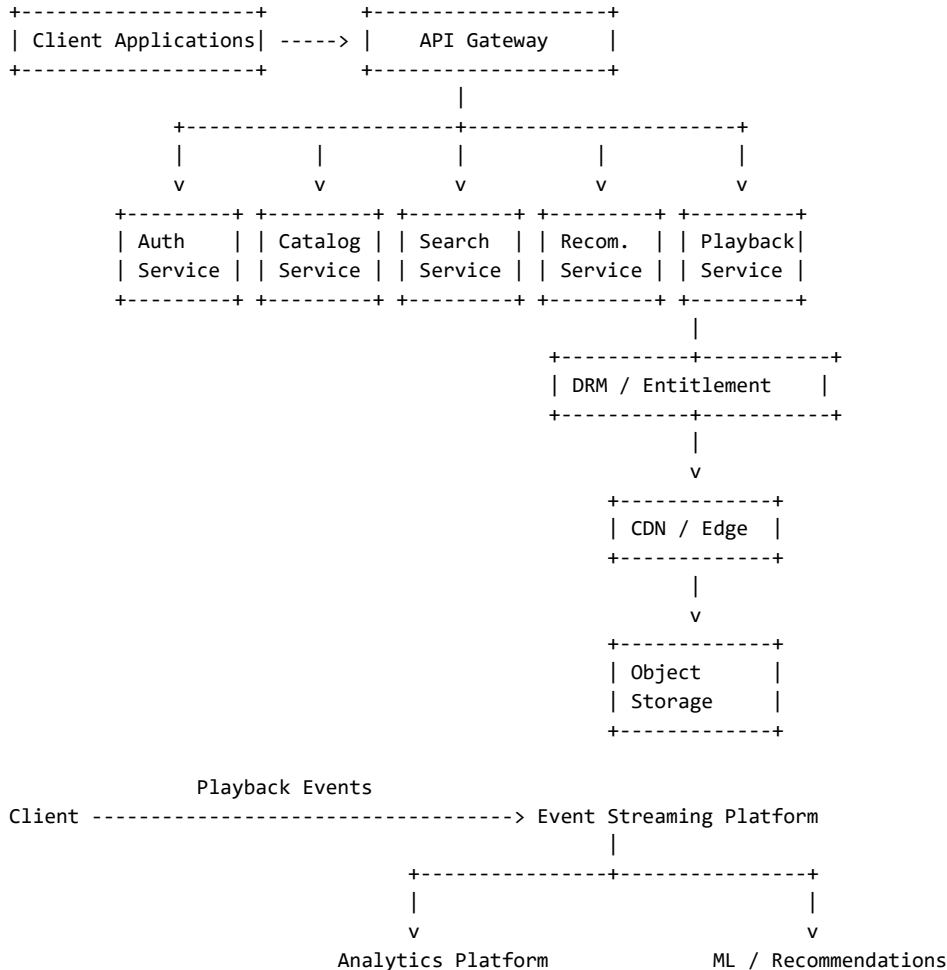
CDN / Edge Network	Geographically dispersed cache that terminates viewer connections close to users, cutting start-up time and origin load.
DRM & License Service	Issues per-device content keys and licenses so premium assets remain encrypted outside authorized players.
User Activity Service	Persists resume bookmarks, play history, ratings, and My List membership for fast per-profile reads.
Event Streaming Platform	Durable bus for domain events such as play, pause, seek, heartbeat, finish, impression, and click.
Analytics & Data Platform	Batch and streaming consumers that power QoE dashboards, A/B analysis, content performance, and model retraining.
Databases & Caches	Purpose-built persistence: OLTP stores for orders and entitlements, NoSQL for massive profile/activity state, search indexes for discovery, and in-memory caches for hot keys.
Monitoring & Observability	Metrics, logs, traces, SLO burn-down, anomaly detection, and paging that keep the whole estate healthy.

7. High-Level Architecture and Data Flow

1. The viewer launches the app and signs in; identity is validated via the API Gateway and Authentication Service.
2. The home screen is assembled from the Catalog Service for rows and artwork plus the Recommendation Service for personalized ordering.
3. Typed queries are fanned out to the Search Service, which returns ranked titles, filters, and suggestions.
4. Selecting a title invokes the Playback Service, which re-checks subscription entitlement and opens a playback session.
5. The session response carries signed manifest URLs together with DRM acquisition details for the player.
6. The player fetches successive segments over adaptive streaming, always requesting the rendition that fits current throughput.
7. Edge PoPs answer from local cache on hits; on misses they pull once from origin, cache, and then serve.
8. Quality is continuously adapted using buffer health, measured bandwidth, and device decoding capability.
9. Lifecycle events — heartbeats, pauses, seeks, and completions — are emitted asynchronously to the event bus.

10. Analytics, experimentation, and model-training consumers subscribe to that bus to refresh insights and future recommendations.

8. Simplified Architecture Diagram



9. Conclusion

A service at Netflix scale succeeds by keeping request handling and byte movement on separate paths. Lightweight, stateless microservices backed by fit-for-purpose stores manage accounts, discovery, and session setup, while a planetary edge cache, adaptive streaming, and asynchronous telemetry carry the heavy lifting of delivery and learning. Together these choices yield horizontal scalability, resilience to partial failures, and consistently fast starts for viewers anywhere in the world.